SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A

Dry Run Evaluation

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

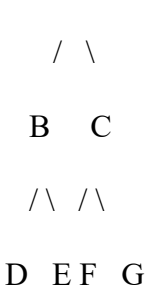
Signature of the student:

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



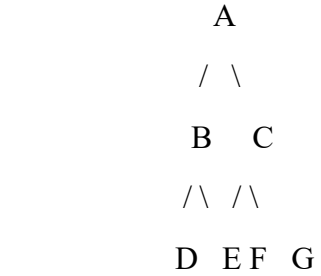
Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration	Queue	Visited Node
1		
2		
3		
4		
5		
6		
7		

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

3. **Initial State :**
   1 3 6
   5 0 2
   4 7 8

Goal State :
   1 2 3
   4 5 6
   7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration Current State Queue Before Expansion Queue After Expansion
1
2
3
4
5

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

1. Breadth-First Search (BFS)

Given graph:



1.1 BFS traversal order

Answer: A → B → C → D → E → F → G

BFS visits nodes level by level using a queue (FIFO).

1.2 Queue contents after each iteration

Iteration	Queue before expansion	Visited node	Queue after expansion
1	A	A	B, C
2	B, C	B	C, D, E
3	C, D, E	C	D, E, F, G
4	D, E, F, G	D	E, F, G
5	E, F, G	E	F, G
6	F, G	F	G
7	G	G	Empty

Conclusion: BFS visits all seven nodes in level order, starting from A.

2. Depth-First Search (DFS)

Using the same graph, start at node A and explore the left branch before the right branch.

DFS traversal order

Answer: A → B → D → E → C → F → G

Step-by-step table

Iteration	Visited node	Stack after visiting
1	A	C, B
2	B	C, E, D
3	D	C, E
4	E	C
5	C	G, F

6	F	G
7	G	Empty

The stack is shown with the top at the right. DFS explores as deeply as possible before backtracking.

Conclusion: DFS visits the left subtree before exploring the right subtree.

3. 8-Puzzle Problem Using BFS

Initial state

1 3 6
5 0 2
4 7 8

Goal state

1 2 3
4 5 6
7 8 0

Here, 0 represents the empty space. BFS explores states level by level using a FIFO queue.

3.1 BFS dry-run table

For compactness, each board is written as three rows separated by /. For example, 136/5_2/478 represents the initial state, with _ denoting the blank.

Iteration	Current state	Queue before expansion	Queue after expansion
1	136/5_2/478	136/5_2/478	1_6/532/478, 136/_52/478, 136/52_/478, 136/572/4_8
2	1_6/532/478	1_6/532/478, 136/_52/478, 136/52_/478, 136/572/4_8	136/_52/478, 136/52_/478, 136/572/4_8, _16/532/478, 16_/532/478
3	136/_52/478	136/_52/478, 136/52_/478, 136/572/4_8, _16/532/478, 16_/532/478	136/52_/478, 136/572/4_8, _16/532/478, 16_/532/478, _36/152/478, 136/452/_78
4	136/52_/478	136/52_/478, 136/572/4_8, _16/532/478, 16_/532/478, _36/152/478, 136/452/_78	136/572/4_8, _16/532/478, 16_/532/478, _36/152/478, 136/452/_78, 13_/526/478, 136/528/47_
5	136/572/4_8	136/572/4_8, _16/532/478, 16_/532/478, _36/152/478, 136/452/_78, 13_/526/478, 136/528/47_	_16/532/478, 16_/532/478, _36/152/478, 136/452/_78, 13_/526/478, 136/528/47_, 136/572/_48, 136/572/48_

Note: This table uses a fixed order for generating neighboring states. The goal is not reached within the first five iterations; BFS continues until it is found.

3.2 Which node is expanded in each iteration?

The expanded states for the first five iterations are:

136/5_2/478

1_6/532/478

136/_52/478

136/52_/478

136/572/4_8

3.3 How many states are generated in total?

Using the successor order in the table, 456 distinct states are discovered, including the initial and goal states, before the goal is reached.

The number of states discovered can vary with the order in which neighboring states are generated.

3.4 Complete solution path

The shortest solution requires 8 moves of the blank space.

1 3 6
5 0 2
4 7 8

Initial state

1 3 6
5 2 0
4 7 8

Move blank right

1 3 0
5 2 6
4 7 8

Move blank up

1 0 3
5 2 6
4 7 8

Move blank left

1 2 3
5 0 6
4 7 8

Move blank down

1 2 3
0 5 6
4 7 8

Move blank left

1 2 3
4 5 6
0 7 8

Move blank down

1 2 3
4 5 6
7 0 8

Move blank right

1 2 3
4 5 6
7 8 0

Move blank right — Goal

3.5 Why does BFS always find the shortest solution in an unweighted 8-puzzle?

BFS explores all states at depth 0, then depth 1, depth 2, and so on. Each legal move has equal cost, so the first time BFS reaches the goal, it has used the minimum possible number of moves. Therefore, BFS guarantees a shortest solution when the goal is reachable.

Conclusion

BFS and DFS traverse graphs using different strategies: BFS explores level by level using a queue, while DFS explores deeply using a stack. In the 8-puzzle problem, BFS systematically explores possible board configurations and finds the shortest solution in eight moves.